Table 1: Path families (one-way delay; the proxy adds the same delay to ACKs). Each seed draws uniformly from the ranges. Cross traffic is a synthetic on/off process at the shared bottleneck (proxy backend only).

family	ranges
dc_microburst	0.25–1 ms; 8–12 MB/s; 20–40 kB buffer; no random loss; 1–3 ms line-rate bursts every 20–40 ms
enterprise_campus	2–8 ms, 0–2 ms jitter; 2.5–5 MB/s; 60–120 kB buffer; 0–0.1% loss; on/off cross traffic
cellular_highlat	40–120 ms, 10–40 ms jitter; 150–600 kB/s; 20–50 kB buffer; 0.5–3% loss

1 Evaluation at scale

1.1 Sample size, path variance and statistics

Earlier tables in this paper used $n = 5$ runs per cell on hand-picked paths. That is too few to support the percentile columns (with $n = 5$ the p95 and p99 are both essentially the sample maximum) and says nothing about how results move with the path. We therefore re-ran the transfer comparison as a multi-seed, multi-path matrix. Three *path families* are defined as parameter ranges (Table 1); seed $s \in \{0, \ldots, 29\}$ draws one concrete path from each family, and every policy is run on exactly that path (a paired design) in a seed-dependent random order, so slow drift in machine load cannot favour one policy. Each cell therefore has $n = 30$ independent runs of a 1 MB transfer, each on a different path, and the interval estimates include both path variance and run noise.

For each cell we report the mean with a t-based 95% confidence interval, the median and p95 with 95% percentile-bootstrap intervals (4000 resamples), and p99 as a point estimate only: with $n = 30$ it interpolates between the two largest observations and has no meaningful interval. Policies are compared by the *ratio* of completion times on the same seed; we report the geometric mean ratio with a 95% interval from a t-interval on the log ratios, a paired t-test and, in the CSV, a Wilcoxon signed-rank test, with Holm correction within each family and reference. Runs that fail the byte-for-byte check are counted (n_{ok}/n) and excluded from the timing statistics; a pair is dropped when either member failed.

1.2 Results: UDP-impairment-proxy backend ($n = 30$ per cell)

Versus CUBIC running over RUDP. on `cellular_highlat` Smart-RUDP's completion time is 0.54× that of the reference (95% CI 0.45–0.63, 27 of 30 paired seeds faster; Holm $p < 0.001$): faster; on `dc_microburst` Smart-RUDP's completion time is 1.36× that of the reference (95% CI 1.17–1.57, 6 of 30 paired seeds faster; Holm $p < 0.001$): slower; on `enterprise_campus` Smart-RUDP's completion time is 0.87× that of the reference (95% CI 0.81–0.94, 24 of 30 paired seeds faster; Holm $p = 0.004$): faster.

1.3 Results: kernel TCP under Linux netem ($n = 30$ per cell)

This backend replaces the user-space proxy with `tc netem` and `tbf` on the loopback device and adds native kernel TCP (CUBIC and BBR) as baselines. Kernel TCP sends the same random payload, over the same bottleneck, with its timer started after `connect()` (the RUDP timer starts after its handshake) and stopped when the receiver has all bytes and answers, which is the same end condition as the RUDP client. Netem impairs both directions of `lo`, whereas the proxy impairs the forward path only, so **the two backends are never mixed in one table**. The synthetic cross-traffic bursts of the proxy have no netem counterpart; under netem `dc_microburst` keeps only its shallow buffer, delay and rate.

PENDING: the kernel-TCP netem scale matrix has not been run yet. Run `execute_wsl_benchmarks.sh` on WSL2/Linux and regenerate with `gen_paper_tables.py`.

PENDING: the kernel-TCP netem scale matrix has not been run yet. Run `execute_wsl_benchmarks.sh` on WSL2/Linux and regenerate with `gen_paper_tables.py`.

PENDING: the kernel-TCP netem scale matrix has not been run yet. Run `execute_wsl_benchmarks.sh` on WSL2/Linux and regenerate with `gen_paper_tables.py`.

PENDING: the netem profile matrix has not been run yet. Run `execute_wsl_benchmarks.sh` on WSL2/Linux and regenerate with `gen_paper_tables.py`.

1.4 Coexistence with $n = 30$ runs per cell

The fairness cells of Section 1 were repeated with 30 runs each (subject against k CUBIC or BBR background flows, $k \in \{1, 2\}$).

Reading Table 4. With 30 runs per cell (Welch tests, unadjusted) the True Smart agent has a higher Jain index than the static agent against CUBIC ($k = 1$: 0.974 vs 0.942, $p = 0.012$; $k = 2$: 0.958 vs 0.881, $p < 0.001$), and than the static agent with the `--polite` guard ($k = 1$: $p = 0.001$; $k = 2$: 0.958 vs 0.919, $p = 0.015$); at $n = 5$ the guard comparison was not significant. Against BBR the picture is different: True Smart's Jain index is not distinguishable from the static agent's ($k = 1$: $p = 0.34$; $k = 2$: $p = 0.33$), and it takes only about 0.74× its equal share, i.e. it yields to the simplified BBR background flows rather than being unfair to them, at the cost of its

Table 2: Completion time of a 1000 kB transfer over randomly drawn paths, $n = 30$ seeds per cell (UDP impairment proxy). Mean with 95% t-interval; median and p95 with 95% bootstrap interval; p99 is a point estimate. n_{ok} = runs delivered byte-identically.

Path family	Policy	n_{ok}	mean [95% CI]	median [95% CI]	p95 [95% CI]	p99
dc_microburst	aimd	30/30	331 [296, 366]	320 [286, 376]	464 [425, 537]	522
	cubic	30/30	377 [306, 448]	347 [278, 407]	659 [490, 1080]	959
	bbr	30/30	236 [204, 268]	217 [180, 276]	371 [304, 486]	456
	neural	30/30	486 [403, 569]	428 [390, 459]	873 [554, 1277]	1161
	smart	30/30	481 [427, 534]	467 [436, 514]	716 [562, 936]	885
	smart+fec	30/30	503 [437, 568]	470 [437, 524]	738 [597, 1192]	1061
enterprise_campus	aimd	30/30	433 [387, 479]	432 [376, 456]	666 [533, 711]	704
	cubic	30/30	423 [378, 468]	406 [340, 467]	647 [508, 778]	759
	bbr	30/30	426 [390, 463]	420 [370, 477]	584 [498, 652]	636
	neural	30/30	392 [336, 448]	386 [334, 413]	516 [462, 1052]	898
	smart	30/30	366 [333, 398]	357 [320, 407]	501 [438, 611]	580
	smart+fec	30/30	389 [355, 423]	375 [328, 428]	524 [486, 595]	579
cellular_highlat	aimd	30/30	12960 [11084, 14835]	12124 [10756, 14523]	20867 [18169, 23929]	23191
	cubic	30/30	13563 [10897, 16229]	12130 [9566, 14691]	28763 [20077, 30710]	30547
	bbr	30/30	8029 [6601, 9456]	7327 [5993, 8269]	12517 [9861, 23959]	20678
	neural	30/30	7729 [6734, 8724]	7266 [6050, 8466]	12477 [9861, 14542]	13960
	smart	30/30	6682 [5934, 7430]	6627 [5555, 7721]	10100 [8556, 10266]	10226
	smart+fec	30/30	5186 [4727, 5644]	5109 [4460, 5718]	7263 [6295, 7870]	7764

Table 3: Paired completion-time ratio versus the reference (same seed ⇒ same path). Geometric mean with 95% t-interval on the log ratio; Holm-adjusted paired t p-value within each family and reference. Ratio < 1: faster than the reference.

Family	Policy	vs	ratio [95% CI]	faster in	p_{Holm}
cellular_highlat	aimd	cubic	1.01 [0.86, 1.18]	12/30	0.917
cellular_highlat	bbr	cubic	0.62 [0.52, 0.75]	24/30	<0.001
cellular_highlat	neural	cubic	0.61 [0.51, 0.74]	25/30	<0.001
cellular_highlat	smart	cubic	0.54 [0.45, 0.63]	27/30	<0.001
cellular_highlat	smart+fec	cubic	0.42 [0.35, 0.51]	29/30	<0.001
dc_microburst	aimd	cubic	0.93 [0.79, 1.11]	18/30	0.425
dc_microburst	bbr	cubic	0.65 [0.55, 0.78]	24/30	<0.001
dc_microburst	neural	cubic	1.32 [1.12, 1.56]	9/30	0.004
dc_microburst	smart	cubic	1.36 [1.17, 1.57]	6/30	<0.001
dc_microburst	smart+fec	cubic	1.41 [1.21, 1.63]	4/30	<0.001
enterprise_campus	aimd	cubic	1.02 [0.93, 1.13]	17/30	1.000
enterprise_campus	bbr	cubic	1.02 [0.93, 1.12]	13/30	1.000
enterprise_campus	neural	cubic	0.91 [0.84, 1.00]	20/30	0.177
enterprise_campus	smart	cubic	0.87 [0.81, 0.94]	24/30	0.004
enterprise_campus	smart+fec	cubic	0.93 [0.85, 1.01]	21/30	0.233

own throughput. We make no fairness claim against BBR beyond "not a bully".

Kernel-TCP retransmission counts come from a system-wide counter and are only meaningful on an otherwise idle host.

1.5 Threats to validity

(i) The path families are our own parameter ranges, not measurements of deployed networks. (ii) Intervals describe sampling error over our families, not any particular real path. (iii) On `dc_microburst` a 1 MB transfer ends within a few round-trips, so handshake and slow start, which are fixed rules shared by all RUDP policies, dominate the comparison. (iv) The learned policies were trained on a simulator with simplified CUBIC and BBR; no result here should be read as evidence about real BBR deployments. (v) A single machine with two cores runs sender, receiver and bottleneck emulation; the order randomisation limits, but does not remove, load-dependent timing noise. (vi)

Table 4: Coexistence, mean ± s.d. over 30 runs per row. J: Jain index over all flows; "share": subject throughput divided by the equal share; "min": smallest flow's share divided by the equal share.

subject	background	k	n	J	share	min
neural	BBR	1	30	0.908 ± 0.082	0.90 ± 0.32	0.71 ± 0.17
neural_polite	BBR	1	30	0.859 ± 0.073	0.67 ± 0.26	0.60 ± 0.13
smart_fair	BBR	1	30	0.926 ± 0.061	0.74 ± 0.13	0.74 ± 0.13
neural	BBR	2	30	0.892 ± 0.095	1.01 ± 0.30	0.62 ± 0.22
neural_polite	BBR	2	30	0.856 ± 0.093	0.71 ± 0.16	0.62 ± 0.16
smart_fair	BBR	2	30	0.868 ± 0.091	0.74 ± 0.18	0.59 ± 0.16
neural	CUBIC	1	30	0.942 ± 0.061	1.21 ± 0.15	0.77 ± 0.12
neural_polite	CUBIC	1	30	0.914 ± 0.089	0.87 ± 0.30	0.74 ± 0.20
smart_fair	CUBIC	1	30	0.974 ± 0.027	0.90 ± 0.13	0.86 ± 0.09
neural	CUBIC	2	30	0.881 ± 0.049	1.47 ± 0.17	0.65 ± 0.10
neural_polite	CUBIC	2	30	0.919 ± 0.069	0.79 ± 0.28	0.67 ± 0.14
smart_fair	CUBIC	2	30	0.958 ± 0.049	0.99 ± 0.13	0.78 ± 0.16